

Machine Learning-Based Detection of Simulated Malware in FPGA Bitstreams

Motivation

- ❖ The increasing reliance on Field-Programmable Gate Arrays (FPGAs) in reconfigurable and cloud-scale systems, valued for their balance of speed, power efficiency, and adaptability, has raised serious concerns about security and trust. Major providers like Microsoft and IBM now deploy FPGAs in performance-critical applications, yet their runtime reconfigurability opens new attack surfaces.
- ❖ In time-based multitенancy scenarios, where users share and reconfigure the same FPGA at different intervals, enforcing trusted execution becomes difficult. Malicious configuration bitstreams can trigger hardware Trojans, enabling denial-of-service (DoS), power abuse, data leakage, and covert channel attacks.
- ❖ Traditional defenses rely on vendor-controlled synthesis flows, which compromise design privacy, or on open hardware ecosystems that still lack safeguards against subtle, low-level threats. As an alternative, this project investigates **non-intrusive, real-time detection of hardware Trojans at the bitstream level** using machine learning.
- ❖ Targeting the Xilinx PYNQ-Z1 platform—an embedded FPGA board with integrated ARM processors—we demonstrate that static analysis of bitstreams using byte-level features enables effective Trojan detection. Our approach does not require netlist access or reverse engineering, making it scalable, private, and practical for secure FPGA deployment workflows.

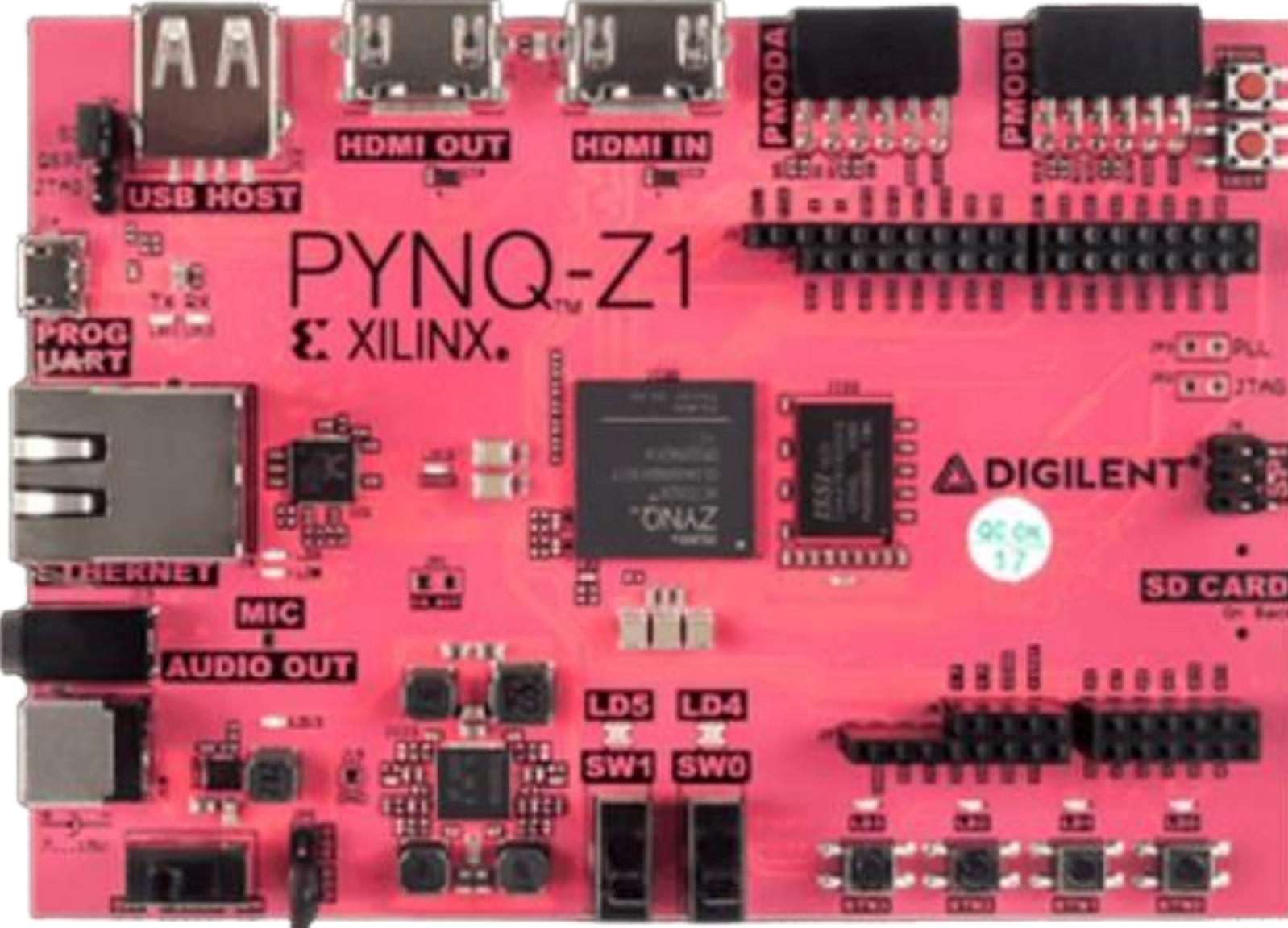


Figure 1. Xilinx PYNQ-Z1 Board Used for On-Device Trojan Detection and Inference.

Method

- ❑ The overall methodology is outlined in **Figures 2 and 3**, which illustrate the end-to-end pipeline used for real-time bitstream-level Trojan detection.
- ❑ Starting with benchmark designs sourced from Trust-Hub, AES-128 and RS232 variants were synthesized into .bit files and labeled as benign, malicious, or empty.
- ❑ Byte-frequency features were extracted from each file, followed by dimensionality reduction using Truncated Singular Value Decomposition (TSVD) to retain structural relevance while improving computational efficiency.
- ❑ To address dataset imbalance, the Synthetic Minority Oversampling Technique (SMOTE) was applied to oversample minority classes before training.
- ❑ Multiple classifiers were evaluated using k-Fold Cross Validation (kFCV). The best-performing model was serialized and deployed to the PYNQ-Z1, where it performed on-device inference using the same preprocessing pipeline to generate predictions.
- ❑ This final step validated the feasibility of real-time Trojan detection in a constrained embedded environment without reliance on external computation.

References

- [1] Elnaggar, R., Chaudhuri, J., Karri, R., & Chakrabarty, K. (2023). Learning Malicious Circuits in FPGA Bitstreams. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(3), 726–739. <https://ieeexplore.ieee.org/document/9828544/>
- [2] Hayashi, V. T., & Vicente Ruggiero, W. (2025). Hardware Trojan Detection in Open-Source Hardware Designs Using Machine Learning. *IEEE Access*. <https://ieeexplore.ieee.org/document/10904479/>
- [3] Pedregosa, F., et al. (2011). scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://dl.acm.org/doi/10.5555/1953048.2078195/>

Training and Deployment

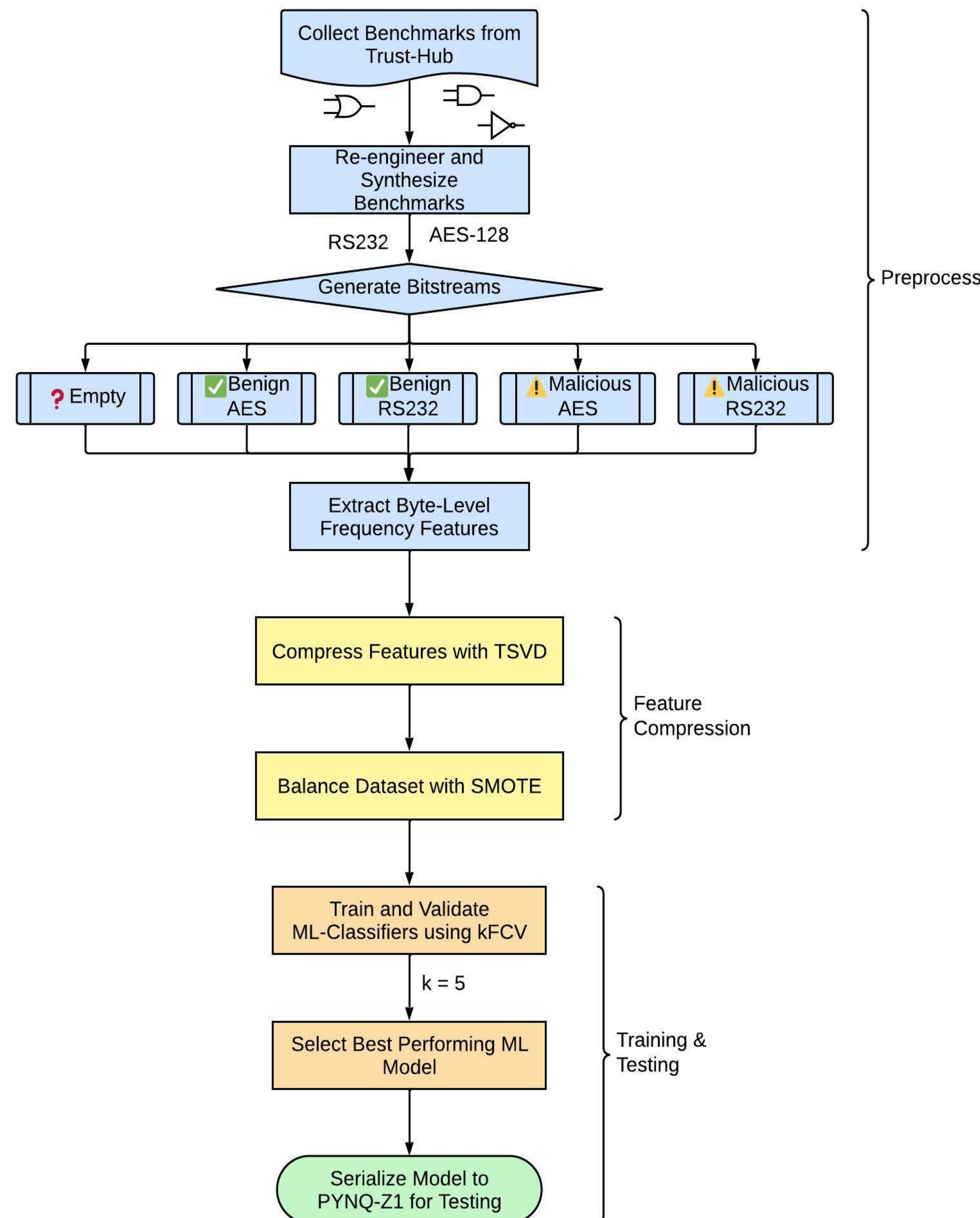


Figure 2. Training and Deployment Pipeline for Bitstream-Level Trojan Detection.

Runtime Prediction

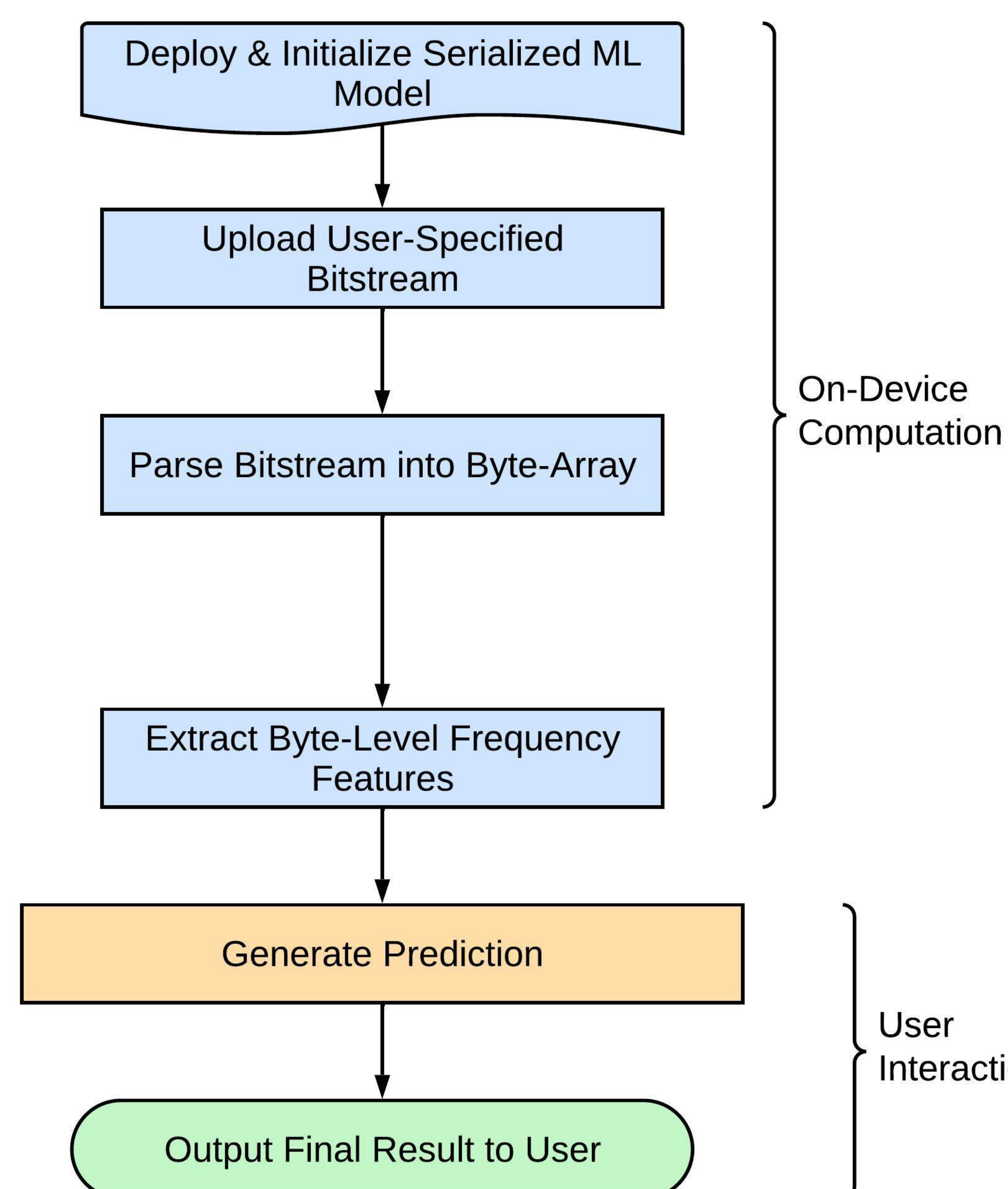


Figure 3. Runtime Prediction Pipeline on PYNQ-Z1 via Jupyter Notebook UI.

Results

Table 1. Performance Metrics (Accuracy, Precision, Recall, F1 Score) for Evaluated Machine Learning Models.

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	0.98 ± 0.02	0.99 ± 0.02	0.98 ± 0.02	0.98 ± 0.02
Gradient Boosting	0.90 ± 0.05	0.88 ± 0.11	0.90 ± 0.06	0.88 ± 0.09
AdaBoost	0.89 ± 0.05	0.87 ± 0.10	0.89 ± 0.05	0.87 ± 0.08
Logistic Regression	0.33 ± 0.07	0.14 ± 0.03	0.36 ± 0.08	0.20 ± 0.05
Naive Bayes	0.95 ± 0.06	0.96 ± 0.05	0.96 ± 0.05	0.96 ± 0.05
SVM (RBF)	0.46 ± 0.10	0.30 ± 0.13	0.51 ± 0.11	0.36 ± 0.12
KNN	0.86 ± 0.04	0.87 ± 0.04	0.86 ± 0.03	0.86 ± 0.04
Decision Tree	0.95 ± 0.00	0.96 ± 0.00	0.95 ± 0.00	0.95 ± 0.00

Table 2. Inference Performance and Classification Results Across Five Deployment Trials on PYNQ-Z1.

Trial	Actual Class	Predicted Class	Load Time (ms)	Feature Extraction Time (ms)	Prediction Time (ms)
1	Malicious RS232	Malicious RS232	239.48	3,157.72	15.03
2	Malicious RS232	Malicious RS232	190.72	3,268.73	16.94
3	Benign AES	Benign AES	23.65	3,167.01	16.81
4	Benign AES	Benign AES	21.85	3,156.55	16.96
5	Malicious AES	Malicious AES	189.02	3,228.77	15.85

Conclusion

- In this work, we proposed an ML-based framework for classifying FPGA bitstreams using static byte-level features and supervised learning. Our pipeline, which leverages dimensionality reduction and a Random Forest classifier, achieved robust multi-class detection performance across both benign and malicious bitstreams.
- On a hold-out test set, the system reached a TPR of 97.14% and a low FPR of 0.8%, indicating strong reliability in accurately identifying malicious designs while minimizing false alarms.
- The model was successfully deployed on the PYNQ-Z1 embedded platform with an average latency of 3.35 seconds per classification, demonstrating the feasibility of real-time Trojan detection on resource-constrained systems. Future work will aim to refine the model to handle more evasive threat patterns and further reduce FPR without sacrificing detection performance.

Acknowledgements

This work was supported by the McNair Junior Fellowship and Office of Undergraduate Research at the University of South Carolina. The authors used OpenAI's ChatGPT to assist with language and grammar correction. While this project utilizes benchmark designs from Trust-Hub, a resource sponsored by the National Science Foundation (NSF), all technical content and analysis were independently developed by the authors. This research also made use of the PYNQ-Z1 FPGA platform, provided by AMD and Xilinx, whose tools and hardware enabled the synthesis and deployment stages of this study.

